Same Signal, Opposite Meaning: Why Adaptive Compute Fails Across Environments

Anonymous

Abstract

Test-time compute improves decision quality for LLM agents, but deciding when it is beneficial remains unresolved. Existing gating methods rely on uncertainty signals, implicitly assuming a fixed relationship between signal value and compute benefit. We show that this assumption is incorrect. Across six diverse agent environments and three model backbones, we find that the same signal value that predicts rollout benefit in one environment predicts rollout harm in another, indicating a reversal in the signal–utility relationship. We trace this to a conflation of two uncertainty sources, *information poverty* and *decision difficulty*, and formalize it via a two-source model grounded in Simpson's paradox, where subgroup-level trends reverse upon aggregation. We show that any gating strategy with a fixed signal–utility direction must underperform never-gating in at least one such environment, making direction discovery unavoidable for cross-environment robustness. Motivated by this result, we propose EAAG (Environment-Aware Adaptive Gating), which learns environment-specific signal–utility relationships through direction discovery. Across six environments, EAAG outperforms all fixed-direction baselines with zero additional per-step overhead, achieving adaptive gating behavior solely by learning signal direction.

1 Introduction

Large language models (LLMs) have rapidly advanced across a wide range of tasks, from question answering and code generation to web navigation and embodied decision making (Yao et al., 2023b; Shinn et al., 2023; Zhou et al., 2024). Yet, many tasks remain beyond the reach of a single forward pass: complex reasoning, multi-step planning, and open-ended exploration often require the model to evaluate alternatives, verify intermediate results, or search over candidate actions (Yao et al., 2023a; Hao et al., 2023; Snell et al., 2024). This has motivated *test-time compute*, where additional computation at inference time improves decision quality. Across multiple benchmarks, test-time optimization allows smaller models to match or exceed the performance of larger ones at lower total cost (Snell et al., 2024; Wang et al., 2026).

While effective, test-time compute is expensive, incurring substantial per-step overhead (e.g., requiring multiple rollouts or candidate evaluations at each decision point) (Yao et al., 2023a; Zhou et al., 2024). As agents are deployed across diverse settings, applying this overhead uniformly becomes wasteful when only a fraction of steps benefit. This has motivated *adaptive gating*: using uncertainty signals to decide *when* extra compute is worthwhile. Prior work explores diverse mechanisms, including entropy thresholding (Lee et al., 2025; Huang et al., 2026; Jin et al., 2026), calibrated confidence (Muennighoff et al., 2025; Han et al., 2025), vote disagreement (Lee et al., 2026), and learned difficulty estimators based on hidden states or reinforcement learning (Liu et al., 2025; Zhu et al., 2025; Zhang et al., 2025; Fang et al., 2025). Across these mechanisms, adaptive gating consistently reduces cost while preserving or improving task performance within each method's evaluation setting.

However, this consensus rests on a narrow empirical base. The fixed-direction assumption holds well in the settings where these methods have been developed and evaluated, predominantly homogeneous reasoning benchmarks where signal direction is consistent by construction. Whether it transfers to heterogeneous agent environments is an open question prior work has not directly addressed, and we show that it does not: the relationship between uncertainty and compute value can reverse direction across environments, and adaptive gating can actively underperform never invoking the optimizer. This raises a central question:

what determines the relationship between uncertainty signals and the value of computation, and can this relationship be learned in a way that generalizes across environments?

We answer this through systematic measurement across six agent environments spanning QA (Yang et al., 2018), code generation (Hendrycks et al., 2021), web navigation (Yao et al., 2022), fact verification (Thorne et al., 2018), text games (Jansen and Côté, 2023), and crafting planning, using three model backbones. The signal–utility relationship reverses direction across environments: the same signal that predicts rollout benefit in code generation predicts rollout harm in fact verification, and can further reverse across model backbones within the same environment. Moreover, the most informative signal itself varies across environments, with structural features dominating in some settings while entropy carries little or no information in others.

We trace this reversal to a conflation of two distinct sources of uncertainty. *Information poverty* (Type I) arises when the agent lacks sufficient information; rollouts from the same model cannot compensate, and high uncertainty predicts *negative* value of computation. In contrast, *decision difficulty* (Type D) arises when multiple viable options exist; additional computation helps explore alternatives, and high uncertainty predicts *positive* value. Code generation often involves multiple valid implementations (Type D), while fact verification may lack sufficient evidence altogether (Type I). Because environments differ in their mixture of these two state types, the same uncertainty signal carries opposite meanings depending on which source dominates, a phenomenon closely related to Simpson's paradox (Simpson, 1951; Wang et al., 2023). The consequence is sharp: no fixed-direction gating strategy can avoid net-harmful gating across environments with opposing signal directions (Proposition 1), and once the direction is misaligned, improved calibration can *worsen* performance by more precisely selecting harmful states.

Motivated by this analysis, we propose **EAAG** (**E**nvironment-**A**ware **A**daptive **G**ating), a deliberately minimal method that learns environment-specific signal–utility relationships from interaction data. Since the primary challenge is identifying signal direction rather than modeling capacity, EAAG's contribution is not a new gate architecture but a pipeline that reduces the adaptive-gating problem to a form where a standard sparse linear classifier suffices, adding zero per-step cost at deployment. Across six environments and three LLM backbones, EAAG outperforms all fixed-direction baselines.

Taken together, these results point to a more fundamental observation: uncertainty signals like token entropy are not sufficient statistics for the value of computation, because their meaning depends on a latent state property that the signal alone cannot identify. Direction discovery is the minimal mechanism that closes this gap.

Our main contributions are as follows:

- **A structural limitation of uncertainty-based gating.** We show that uncertainty signals are not sufficient statistics for the value of computation: their meaning depends on a latent state type, and as a consequence no fixed-direction gate can be universally correct.
- **Empirical finding.** We demonstrate systematic reversal of signal–utility direction across environments and model backbones, and show that the most informative signals vary across settings.
- **Adaptive gating method.** We propose EAAG, which learns signal direction per environment instead of assuming it, outperforming all fixed-direction baselines across six environments with zero deployment overhead.

2 Related Work

Prior work on adaptive test-time compute spans diverse mechanisms but rests on a shared assumption: that uncertainty signals have a fixed, environment-independent relationship with compute value.

Gating on uncertainty signals. One line of work allocates extra computation based on token confidence, entropy, or disagreement, under the assumption that higher uncertainty indicates greater need for compute. In the reasoning setting, signal-based methods extend chain-of-thought generation when token confidence or entropy exceeds a threshold (Lee et al., 2025; Huang et al., 2026; Jin et al., 2026; Sharma and Chopra, 2025; Muennighoff et al., 2025; Han et al., 2025). In the agent setting, vote-based methods (Lee et al., 2026) sample multiple candidate actions and invoke an arbiter when they disagree, while verbalized uncertainty methods (Zhang et al., 2026) prompt the agent to reflect when its self-reported

confidence is low.

Gating on learned difficulty. A second line learns to predict task difficulty and allocate compute accordingly. Hidden-state probes (Liu et al., 2025; Zhu et al., 2025) estimate difficulty from internal representations, and RL-based methods (Zhang et al., 2025; Fang et al., 2025; Aggarwal and Welleck, 2025; Paglieri et al., 2025) learn think/no-think policies end-to-end. Though these methods do not commit to a specific signal, their gating policies effectively behave as if higher difficulty warrants more compute, sharing the same fixed, environment-independent direction as the first family. This assumption holds well in the settings where these methods have been developed and evaluated, predominantly homogeneous reasoning benchmarks such as MATH, GSM8K, and GPQA, where signal direction is consistent by construction. Whether it transfers to heterogeneous agent environments is an open question that prior work has not directly addressed; we show it does not (Proposition 1). A per-method comparison is in Appendix A.

Orthogonal: search-based optimizers. Search methods such as Tree of Thoughts (Yao et al., 2023a), LATS (Zhou et al., 2024), RAP (Hao et al., 2023), and FLARE (Wang et al., 2026) improve *what* the optimizer does through tree search, value propagation, and lookahead evaluation. These are orthogonal to our work: we control *when* to invoke the optimizer, and our gate composes with any such procedure.

Metareasoning and uncertainty semantics. Rational metareasoning (Russell and Wefald, 1991; Sabbata et al., 2024) formalizes the value of computation (VOC), classically assuming VOC ≥ 0 since an agent can disregard unfavorable computations. We show that this assumption fails in the agent setting, where the same model both proposes and evaluates actions: under this evaluator–executor identity, VOC can be *negative*, making direction discovery necessary. (Tao et al., 2025) and (Heo et al., 2025) find that calibration quality varies across task types, and (Abbasi Yadkori et al., 2024) decouple epistemic and aleatoric uncertainty—providing converging evidence that uncertainty semantics are domain-dependent, though they stop short of documenting direction reversal. Our two-source model formalizes this domain dependence (§3.2), and our reversal results make it actionable.

Table 1: Signal–utility correlations across 6 environments and 3 backbones. ρ: Spearman correlation of token entropy with optimizer utility U. *: significant at $p<0.05$. Direction varies across *both* environments and model backbones.

Environment	Qwen3	Phi-3.5	Llama-3.1	Cons.
HotpotQA	-0.327^*	$+0.184^*$	-0.346^*	✗
WebShop	$+0.133^*$	$+0.335^*$	$+0.287^*$	✓
FEVER	-0.119^*	-0.156^*	$+0.428^*$	✗
TWExpress	-0.290^*	$+0.000$	$+0.000$	—
Plancraft	-0.016	$+0.000$	-0.176^*	—
APPS	$+0.317^*$	-0.024	-0.249^*	✗

3 Why Fixed-Direction Gating Fails

We test the fixed-direction assumption by measuring signal–utility correlations across diverse environments. If the assumption holds, correlations should be consistent; if it fails, the failure mode will guide method design.

We study LLM-based agents in multi-step interactive environments. At each step t, the agent observes state s_t, selects action a_t, and transitions to s_{t+1}. An environment-specific *test-time optimizer* T (e.g., rollout evaluation, K-variant sampling) can be invoked at any step to potentially improve action quality. We define the *optimizer utility* as:

$$U(T, s) = \mathbb{E}[R(\tau) \mid a{=}T(s)] - \mathbb{E}[R(\tau) \mid a{=}\pi(s)] \quad (1)$$

where $R(\tau)$ is the episode return. $U > 0$ indicates the optimizer improves the outcome; $U \leq 0$ indicates it is wasteful or harmful. The *adaptive gating problem* is to learn a gate $g : \mathcal{S} \to \{0, 1\}$ that triggers T only when $U > 0$.

3.1 Signal–Utility Direction Reverses Across Environments and Backbones

The signal–utility direction is not fixed. Table 1 reports Spearman correlations between token entropy and optimizer utility across 6 environments and 3 model backbones. Two patterns stand out. First, even within a single backbone, the direction varies across environments: on Qwen3, entropy is negatively correlated with utility in fact-verification environments (FEVER, TWExpress) but positively in code generation (APPS) and web navigation (WebShop), with near-zero correlation in weak-signal environments (Plancraft). Second, the same signal in the same environment can reverse across backbones: in 3

out of 4 environments with multiple significant correlations, models disagree on the sign. Most strikingly, FEVER flips from $\rho{=}{-}0.156$ on Phi-3.5 to $\rho{=}{+}0.428$ on Llama-3.1. Structural signals such as `step_count` are more stable across backbones (Appendix F.4), while entropy is unpredictable without per-environment, per-model measurement. Crucially, this direction mismatch is not benign: as Section 5.2 will show, gates that assume the wrong direction systematically fall *below* the no-trigger baseline, making them strictly worse than doing nothing in mismatched environments.

Signal identity also varies across environments. Beyond direction, the most informative signal itself differs from environment to environment: `step_count` dominates in most environments, but WebShop instead relies on `num_available_actions` and task-specific features (Appendix E.1). Moreover, even the best single signal is near-chance: per-environment AUC analysis shows single token entropy at AUC≈0.50, while multi-signal logistic gates reach AUC≈0.83 and hidden-state probes reach AUC≈0.90 (Appendix E.3). Predicting rollout value requires multiple signals selected per environment, not a single universal scalar.

3.2 Why Does Direction Reverse? A Two-Source Model

The findings above establish that fixed-direction gating fails; we now ask *why*. We trace direction reversal to a minimal mechanism that jointly accounts for the observed patterns: the conflation of two distinct sources of uncertainty.

Intuition. Consider a web-shopping agent choosing between products. When it hesitates (high entropy) because multiple items match the query, exploring alternatives via rollouts is valuable, since the agent has adequate information but faces a genuine choice. Now consider a fact-verification agent that hesitates because the retrieved evidence is ambiguous. Here, high entropy reflects *missing information*, not optionality: rollouts from the same model with the same evidence will not resolve the ambiguity. In the first case, entropy positively predicts rollout utility; in the second, it predicts rollout *futility*. The same numerical signal carries opposite meaning because the *source* of uncertainty differs.

Model. We model direction reversal as arising from two sources of uncertainty at each decision step:

- **Type I (information poverty):** The agent lacks sufficient information. Rollouts from the same model cannot compensate, so utility is *negatively* related to entropy: $U_I(s) \sim -\alpha H(s) + \varepsilon_I$, $\alpha > 0$.
- **Type D (decision difficulty):** The agent has adequate information but faces multiple viable paths. Rollouts exploit this diversity, so utility is *positively* related to entropy: $U_D(s) \sim +\beta H(s) + \varepsilon_D$, $\beta > 0$.

Let $p_I(\mathcal{E})$ denote the fraction of Type I states in environment $\mathcal{E}$. The marginal correlation becomes

$$\rho(\mathcal{E}) \;\approx\; \beta - (\alpha + \beta)\, p_I(\mathcal{E}), \qquad (2)$$

which reverses sign at the critical threshold $p_I^* = \beta/(\alpha + \beta)$. Environments with $p_I > p_I^*$ exhibit negative ρ (Type I dominated); those with $p_I < p_I^*$ exhibit positive ρ (Type D dominated); near p_I^*, the two effects cancel and entropy carries negligible signal. This prediction is consistent with the observed Qwen3 pattern in Table 1: search-based QA (FEVER, HotpotQA, TWExpress) shows negative ρ, code generation (APPS) shows positive ρ, and weak-signal environments (Plancraft) cluster near zero. Multi-backbone results (Appendix F.4) further show that the model backbone modulates the effective p_I, making direction unpredictable from environment structure alone. We treat this two-source decomposition as a *minimal falsifiable* model rather than a post-hoc explanation: in Section 5.4 we derive three predictions from it (temporal dynamics, cross-environment consistency, and signal identity alignment) and verify each on held-out data.

Theoretical grounding. This direction reversal is a special case of Simpson's paradox (Simpson, 1951; Pearl, 2022): aggregating subpopulations with opposing within-group trends reverses the aggregate correlation. Our decomposition adapts the epistemic/aleatoric distinction (Der Kiureghian and Ditlevsen, 2009) to the meta-decision setting: Type I states exhibit epistemic-like uncertainty (information deficit), while Type D states exhibit aleatoric-like diversity (multiple viable paths).

3.3 Necessity of Direction Discovery

Proposition 1 (Necessity of Direction Discovery)**.** *Suppose there exist two environments $\mathcal{E}_1, \mathcal{E}_2$ with opposite true directions:* $d^*(\mathcal{E}_1) = +1$

and $d^*(\mathcal{E}_2) = -1$. *Then no fixed-direction gate* g_d *can simultaneously satisfy* $\mathrm{SR}(g_d, \mathcal{E}_1) \geq \mathrm{SR}(\mathrm{base}, \mathcal{E}_1)$ *and* $\mathrm{SR}(g_d, \mathcal{E}_2) \geq \mathrm{SR}(\mathrm{base}, \mathcal{E}_2)$.

The intuition is direct: a gate with $d = +1$ triggers on high-signal states, which in a Type I environment correspond exactly to states where rollouts harm the agent; the symmetric argument holds for $d = -1$ in Type D environments. Since real-world deployments span both environment types, no fixed-direction strategy can deliver non-negative value of computation universally—direction discovery is a necessary precondition for cross-environment robustness, not a refinement of it. The full proof is in Appendix B.1.

4 Method: Environment-Aware Adaptive Gating

Section 3 imposes three requirements on any gating method that aims for cross-environment robustness. (i) Signal direction must be *learned* per environment, since Proposition 1 rules out any fixed choice. (ii) The feature set must be *multi-signal*, since no single signal is reliably informative across environments (§3.1). (iii) Exploration must be *signal-agnostic*, since conditioning on a signal whose semantics are unknown would bias the collected data in a direction that depends on the answer.

EAAG is a deliberately minimal instantiation of these requirements. Its contribution is not a new gate architecture but a pipeline that reduces the adaptive-gating problem to a form where a standard sparse linear model suffices: randomized exploration (§4.1) handles the unknown prior direction, an optional LLM feature layer (§4.2) proposes task-specific signals when universal ones are inadequate, and a sparse linear gate (§4.3) recovers per-environment direction from the resulting dataset. Algorithm 1 summarizes the full pipeline and its connection to requirements (i)–(iii).

Since the primary bottleneck is signal direction rather than classification capacity (§5.3), we invest complexity in exploration and feature selection and keep the deployed gate deliberately minimal.

4.1 Explore: Signal-Agnostic Data Collection

Exploration collects the (s, U) pairs that will be used to fit the gate. The central constraint is requirement (iii): the collection policy must not condition on any signal whose semantics we are trying to learn. Any signal-dependent alternative, such as uncertainty sampling, would condition on a signal whose relationship to U is exactly what §3.1 showed to be unknown, biasing $\mathcal{D}$ in a direction that depends on the quantity being estimated. Warm-starting from prior-method data inherits the same circularity.

EAAG therefore uses the simplest policy consistent with this constraint. At each step t in episode i, a Bernoulli coin flip determines the gating decision:

$$z_t^{(i)} \sim \mathrm{Bernoulli}(\varepsilon_{\text{explore}}), \quad \varepsilon_{\text{explore}} = 0.5, \quad (3)$$

where $z_t{=}1$ triggers the optimizer T and $z_t{=}0$ uses the base action. Over $N_{\text{explore}} = 50$ episodes, this yields an unbiased dataset

$$\mathcal{D} = \left\{ \left(\phi(s_t^{(i)}),\ U_t^{(i)} \right) : z_t^{(i)} = 1 \right\}_{t,i}, \quad (4)$$

where $U_t^{(i)} \in \{0, 1\}$ indicates whether the optimizer improved the outcome at step t, and $\phi(s)$ is the feature vector described in §4.2.

4.2 Reason: LLM Feature Layer

The next step builds the feature pool $\phi_{\text{cand}}(s)$ that the gate will be fit over. Requirement (ii) demands that this pool be rich enough to include each environment's relevant signals, but not so environment-specific that deployment requires manual engineering per task. EAAG's default is a fixed set of universal features: `step_count`, `token_entropy`, `state_length`, `action_entropy`, and others. These are available in every environment without task-specific engineering. In the environments we study, this pool alone is sufficient to recover a gate that outperforms all fixed-direction baselines (§5.3).

For environments where universal features may be inadequate, such as WebShop, whose gating benefits from task-specific signals like `num_available_actions`, EAAG offers an *optional* automation layer: an LLM is given a summary of $\mathcal{D}$ and proposes task-specific feature hypotheses $\phi_{\text{LLM}}(s)$, which join the universal pool to form

$$\phi_{\text{cand}}(s) = \phi_{\text{univ}}(s) \ \cup \ \phi_{\text{LLM}}(s). \quad (5)$$

The ablation in §5.3 shows its gains concentrate in the few settings where universal features miss the relevant structure, so the LLM is best viewed as an automation convenience for zero-shot deployment rather than a core component the method depends

on.

4.3 Learn: Sparse Linear Gate

Given $\mathcal{D}$ and ϕ_{cand}, EAAG fits an ℓ_1-regularized logistic regression:

$$\mathbf{w}^*, b^* = \arg\min_{\mathbf{w},b} \sum_{(\phi,U)\in\mathcal{D}} \ell\big(\sigma(\mathbf{w}^\top\phi + b),\, U\big) + \lambda\|\mathbf{w}\|_1 \tag{6}$$

where ℓ is the binary cross-entropy loss and λ controls sparsity over ϕ_{cand} so that uninformative features drop out. The deployed gate is

$$g(s) = \mathbf{1}\Big[\sigma(\mathbf{w}^{*\top}\phi(s) + b^*) > \tau\Big], \tag{7}$$

with threshold τ set by cross-validation on $\mathcal{D}$. Gate fitting completes in under a second, and deployment adds zero per-step overhead beyond feature computation. Implementation details are in Appendix C.3.

Signed weights as a diagnostic. The ℓ_1 penalty in Eq. 6 simultaneously performs feature selection and direction discovery. The sign of each fitted weight provides an interpretable per-environment estimate of the Two-Source mixture of §3.2: $w_i > 0$ indicates the environment sits on the Type D side of the reversal threshold p_I^* for signal i (high signal $\rightarrow$ trigger), $w_i < 0$ the Type I side (high signal $\rightarrow$ *don't* trigger), and $w_i \approx 0$ an uninformative signal. This is a diagnostic reading of the fitted model, not a novel modeling choice; any signed linear classifier would expose the same information.

Online adaptation. For drifting environments, EAAG can optionally refit during deployment using ε-greedy exploration ($\varepsilon : 0.1 \rightarrow 0$) and periodic retraining. This mode is *not free*: ε-greedy overrides ignore the current gate, so in Type I environments a fraction of overridden steps will trigger harmful rollouts. For stationary environments, which are the setting of our main experiments, we recommend skipping online adaptation and relying on the offline-fitted gate.

The complete three-phase procedure is summarized in Algorithm 1 (Appendix C.3).

5 Experiments

5.1 Experimental Setup

We evaluate EAAG along three axes: does environment-aware gating Pareto-dominate fixed-direction baselines (§5.2); which components drive the gains (§5.3); and do the Two-Source Model's predictions hold on held-out data (§5.4).

Environments. We evaluate on 6 environments: HotpotQA (Yang et al., 2018), APPS (Hendrycks et al., 2021), WebShop (Yao et al., 2022), FEVER (Thorne et al., 2018), TWExpress (Jansen and Côté, 2023), and Plancraft (Dagan et al., 2024), spanning QA, code generation, web navigation, fact verification, text games, and crafting planning. These cover the full range of the Two-Source Model, from pure Type I (FEVER) through mixed (WebShop) to Type D (APPS), including extreme rollout properties (TWExpress: rollout-safe; Plancraft: rollout-harmful). Full environment specifications are in Appendix C.1.

Baselines. All methods share the same agent and optimizer T; only the gating decision differs. We include three bounds: *base_only*, which never triggers the optimizer; *always_trigger*; and *oracle*. Six fixed-direction baselines all assume that high uncertainty warrants additional compute: CaTS (Huang et al., 2026), SEAG (Lee et al., 2025), CoRefine (Jin et al., 2026), CATTS (Lee et al., 2026), AUQ (Zhang et al., 2026), and s1_budget (Muennighoff et al., 2025). We also evaluate BSW, which intentionally reverses the learned direction to quantify the cost of direction mismatch. Full specifications are in Appendix C.2.

Metrics. SR (success rate, ↑) and Cost (×base, ↓): total deployment tokens normalized by base_only tokens per episode. This metric counts *all* LLM calls—base proposer, gate overhead (CATTS K=5 voting, AUQ confidence queries), and rollout calls—ensuring that per-step overhead is not hidden. Calibration or exploration overhead is excluded for all methods. A method Pareto-dominates another if SR $\geq$ and Cost $\leq$ with at least one strict inequality. Token cost constants are reported in Appendix C.5.

5.2 Main Results

EAAG Pareto-dominates fixed-direction baselines. Figure 1 shows the SR–Cost Pareto frontier across all 6 environments on Qwen3-4B; full numerical results across three backbones are in Table 8 (Appendix D).

Three patterns emerge. First, fixed-direction baselines catastrophically fail when the assumed direction is wrong: on FEVER (Type I),

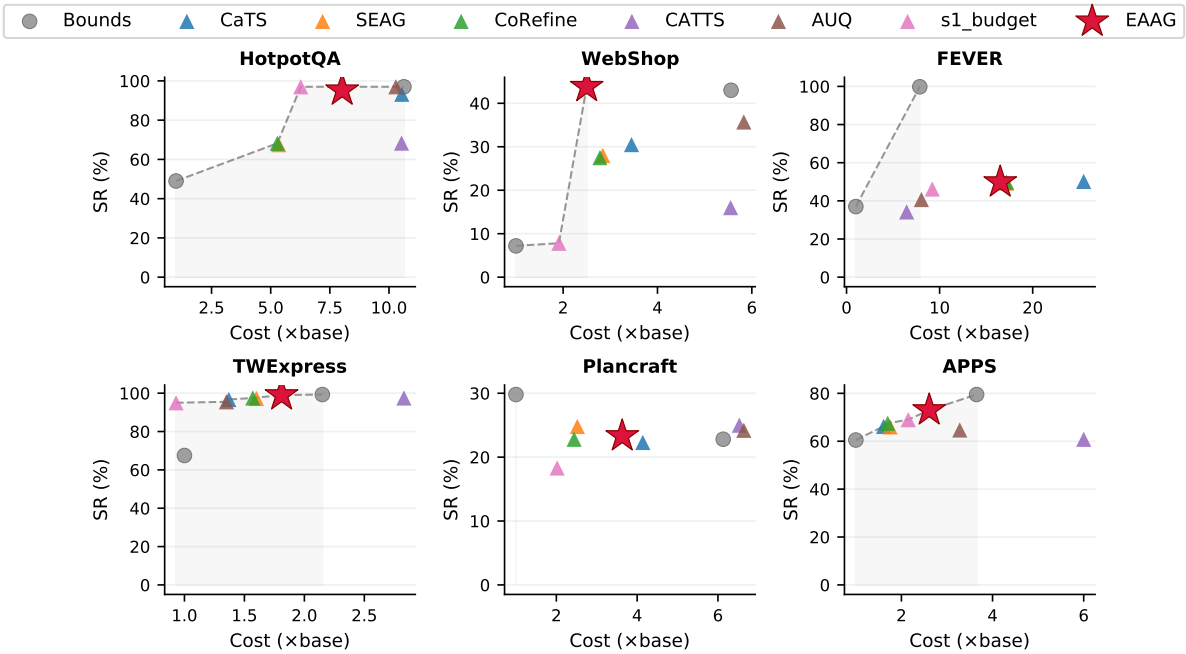

Figure 1: Pareto frontier: SR vs. Cost across all 6 environments (Qwen3-4B). ○ = bounds, △ = fixed-direction baselines, ★ = EAAG. Dashed line: Pareto frontier. EAAG sits on or near the frontier in every environment. Full numerical results across three backbones are in Table 8.

CATTS triggers almost never (SR = 34.2%, below the 37.0% no-trigger baseline) yet still costs 6.46×base due to its K=5 per-step voting overhead. Second, EAAG achieves competitive SR at substantially lower cost than always-triggering. On WebShop, EAAG reaches 43.8% SR at 2.50×base, matching always_trigger (43.0%) at less than half the cost (5.56×). On APPS, EAAG outperforms the best baseline (s1_budget, 69.0%) by 4 pp while using comparable cost. Third, in the rollout-harmful environment Plancraft (Δ=−7.0), EAAG learns to suppress triggering and recovers near base_only performance (23.3% vs. 29.8%), while most baselines that trigger aggressively fall below base_only (s1_budget: 18.3%).

5.3 Ablation Studies

The main results show *that* EAAG works; we now ask *why*. We ablate the two key components: direction learning and the LLM feature layer.

Direction is the primary determinant of gate quality. Intentionally reversing the learned direction (BSW ablation) causes large SR drops in every environment with strong signal: FEVER falls from 49.8% to 13.0% (−36.8 pp), HotpotQA from 95.2% to 58.2% (−37.0 pp), and WebShop from 43.8% to 20.6% (−23.2 pp). In contrast, weak-signal environments like Plancraft ($|\rho|$=0.016) are unaffected (+2.2 pp). The degradation scales with signal strength: wrong direction is not a minor calibration error but a structural failure, as Proposition 1 predicts. Full per-environment results are in Appendix B.2.

Table 2: LLM feature ablation. SR (%) with and without LLM-generated features. The contribution is <1 pp in most environments; FEVER is the exception.

Environment	w/ LLM	w/o LLM	Δ
HotpotQA	95.2	95.8	−0.6
WebShop	43.8	43.7	+0.1
FEVER	49.8	40.7	+9.1
TWExpress	99.0	99.2	−0.2
Plancraft	23.3	27.2	−3.9
APPS	66.0	65.8	+0.2

The LLM feature layer is an automation convenience, not a core component. Table 2 compares EAAG with and without the LLM feature layer. In 4 of 6 environments, removing LLM features changes SR by <1 pp. In Plancraft, LLM features even hurt performance (−3.9 pp). The one environment where the LLM contributes meaningfully is FEVER (+9.1 pp), where task-specific signals extend the universal feature pool. The LLM layer is best viewed as an automation convenience for zero-shot deployment rather than a component the method depends on.

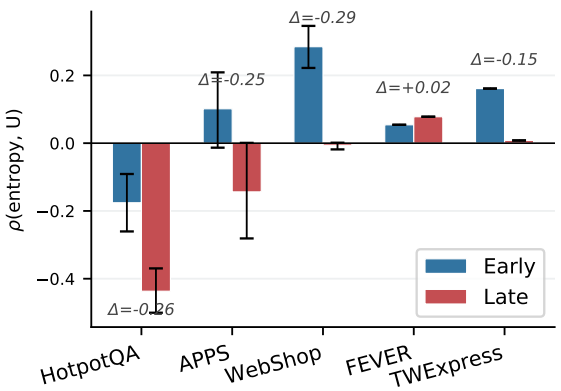

Figure 2: P1 verification: ρ(entropy, U) shifts from early to late steps. Blue: early (step $\leq$ median), red: late. Δ annotations show consistent shift toward the dominant uncertainty type, confirming temporal dynamics prediction. Plancraft is omitted because its entropy signal is too weak ($|\rho|$=0.016) for a meaningful early/late comparison.

Beyond direction and features, EAAG's trigger rate also adapts per environment, ranging from 73% (TWExpress) to <20% (late-step Plancraft). This behavior emerges from direction learning alone and is analyzed in Appendix F.7.

5.4 Two-Source Model Verification

The ablations confirm EAAG's practical advantage; we now test whether the Two-Source Model (§3.2) makes correct predictions on held-out data. Following (Schaeffer et al., 2023), we derive three testable predictions and verify each:

P1: Temporal dynamics. Within the same environment, ρ should *decrease* over the episode: early steps mix Type I and Type D uncertainty, while late steps, after information-gathering attempts, isolate the residual Type I component.

P2: Cross-environment consistency. Environments with similar task structure should exhibit similar ρ (e.g., FEVER $\approx$ HotpotQA, both search-based QA).

P3: Signal identity alignment. In Type I environments, the strongest signal should measure information sufficiency; in Type D environments, decision complexity.

P1: Temporal dynamics. Figure 2 shows that within each environment, ρ shifts between early and late steps. In HotpotQA, ρ moves from -0.176 (early) to -0.437 (late): late-step entropy reflects deeper information poverty after failed evidence retrieval. In WebShop, the strong positive early-phase signal (ρ=+0.285) vanishes in the late phase. In APPS, the positive ρ strengthens over steps as decision complexity accumulates. The pattern holds across all environments with sufficient signal (Appendix F.1).

P2: Cross-environment consistency. Same-family environments share the same direction (Table 1). FEVER and HotpotQA, both search-based QA, share negative ρ; WebShop and APPS, both involving active choice among alternatives, share positive ρ. Magnitude differences within each pair reflect the degree of Type I contamination.

P3: Signal identity alignment. The strongest signal per environment matches the Two-Source Model prediction (Appendix F.2): `step_count` dominates in Type I environments as an information-accumulation proxy, while `num_available_actions` dominates in WebShop as a decision-complexity proxy. A controlled manipulation experiment within HotpotQA further confirms that signal identity shifts with information structure (Appendix F.3).

All three predictions are supported. Additional robustness checks are in Appendix F: multi-backbone verification (§F.4), stratified analysis (§F.5), and gate complexity ablations (§F.6).

6 Limitations

EAAG underperforms on FEVER, where rollout value concentrates in the first one or two steps and random exploration misses these critical steps with 50% probability. This failure mode is itself predicted by the Two-Source Model: FEVER is the most extreme Type I environment, where undirected exploration is least efficient. Addressing this through curriculum-based exploration is an important direction for future work. We verify our findings on three backbones (4B–8B) across all six environments (Appendix F.4); testing on larger models (70B+) remains future work. The Two-Source Model assumes linear conditional relationships and serves as an explanatory framework rather than a precise predictor of ρ values. EAAG requires 50 exploration episodes per new environment, a modest but non-zero cost amortized over subsequent deployment.

7 Conclusion

All existing adaptive gating methods for LLM agents rest on an implicit assumption: that uncertainty signals have a fixed relationship with compute value. We show that this assumption reflects a structural limitation. Uncertainty signals are not sufficient statistics for the value of computation, because their meaning depends on a latent state property (information poverty vs. decision difficulty) that the signal alone cannot identify. Across six environments and three model backbones, signal–utility direction reverses; once misaligned, improved calibration *worsens* performance by more precisely selecting harmful states. A two-source model grounded in Simpson's paradox explains when and why reversal occurs, and a necessity proof establishes that direction discovery is unavoidable for cross-environment robustness. EAAG instantiates this principle with a minimal pipeline that collects unbiased data through randomized exploration and recovers per-environment direction through sparse linear learning, adding zero per-step overhead at deployment. Across all evaluated settings, EAAG dominates fixed-direction baselines, and the full range of gating behavior, from aggressive triggering to near-total suppression, emerges from direction learning alone.

References

Yasin Abbasi Yadkori, Ilja Kuzborskij, and András György. 2024. To believe or not to believe your LLM. *CoRR*.

Pranjal Aggarwal and Sean Welleck. 2025. L1: controlling how long A reasoning model thinks with reinforcement learning. *CoRR*.

Gautier Dagan, Frank Keller, and Alex Lascarides. 2024. Plancraft: an evaluation dataset for planning with LLM agents. *CoRR*.

Armen Der Kiureghian and Ove Ditlevsen. 2009. Aleatory or epistemic? does it matter? *Structural safety*.

Guanting Dong, Hangyu Mao, Kai Ma, Licheng Bao, Yifei Chen, Zhongyuan Wang, Zhongxia Chen, Jiazhen Du, Huiyang Wang, Fuzheng Zhang, Guorui Zhou, Yutao Zhu, Ji-Rong Wen, and Zhicheng Dou. 2025. Agentic reinforced policy optimization. *CoRR*.

Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed Roman, Ahmed A Aly, Beidi Chen, and Carole-Jean Wu. 2024. Layerskip: Enabling early exit inference and self-speculative decoding. In *ACL*.

Gongfan Fang, Xinyin Ma, and Xinchao Wang. 2025. Thinkless: LLM learns when to think. *CoRR*.

Alex Graves. 2016. Adaptive computation time for recurrent neural networks. *CoRR*.

Tingxu Han, Zhenting Wang, Chunrong Fang, Shiyu Zhao, Shiqing Ma, and Zhenyu Chen. 2025. Token-budget-aware LLM reasoning. In *ACL*.

Shibo Hao, Yi Gu, Haodi Ma, Joshua Jiahua Hong, Zhen Wang, Daisy Zhe Wang, and Zhiting Hu. 2023. Reasoning with language model is planning with world model. In *EMNLP*.

Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Andy Zou, Dawn Song, and Jacob Steinhardt. 2021. Measuring coding challenge competence with APPS. In *NeurIPS*.

Juyeon Heo, Miao Xiong, Christina Heinze-Deml, and Jaya Narain. 2025. Do llms estimate uncertainty well in instruction-following? In *ICLR*.

Eric Horvitz. 1987. Reasoning about beliefs and actions under computational resource constraints. In *UAI*.

Chengsong Huang, Langlin Huang, Jixuan Leng, Jiacheng Liu, and Jiaxin Huang. 2026. Cats: Calibrated test-time scaling for efficient llm reasoning. In *ICLR*.

Peter Jansen and Marc-Alexandre Côté. 2023. Textworldexpress: Simulating text games at one million steps per second. In *EACL*.

Chen Jin, Ryutaro Tanno, Tom Diethe, and Philip Teare. 2026. Corefine: Confidence-guided self-refinement for adaptive test-time compute. *CoRR*.

Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, Scott Johnston, Sheer El Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Sam Bowman, Stanislav Fort, and 17 others. 2022. Language models (mostly) know what they know. *CoRR*.

Nicholas Lee, Lutfi Eren Erdogan, Chris Joseph John, Surya Krishnapillai, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. 2026. Agentic test-time scaling for webagents. *CoRR*.

Sungjae Lee, Hyejin Park, Jaechang Kim, and Jungseul Ok. 2025. Semantic exploration with adaptive gating for efficient problem solving with language models. In *ACL*.

Falk Lieder and Thomas L Griffiths. 2020. Resource-rational analysis: Understanding human cognition as the optimal use of limited computational resources. *Behavioral and brain sciences*.

Xiang Liu, Xuming Hu, Xiaowen Chu, and Eunsol Choi. 2025. Diffadapt: Difficulty-adaptive reasoning for token-efficient LLM inference. *CoRR*.

Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel J. Candès, and Tatsunori Hashimoto. 2025. s1: Simple test-time scaling. In *EMNLP*.

Davide Paglieri, Bartlomiej Cupial, Jonathan Cook, Ulyana Piterbarg, Jens Tuyls, Edward Grefenstette, Jakob Nicolaus Foerster, Jack Parker-Holder, and Tim Rocktäschel. 2025. Learning when to plan: Efficiently allocating test-time compute for LLM agents. *CoRR*.

Judea Pearl. 2022. Comment: Understanding simpson's paradox. In *Probabilistic and Causal Inference: The Works of Judea Pearl*.

David Raposo, Samuel Ritter, Blake A. Richards, Timothy P. Lillicrap, Peter Conway Humphreys, and Adam Santoro. 2024. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *CoRR*.

Stuart J. Russell and Eric Wefald. 1991. *Do the right thing - studies in limited rationality*. MIT Press.

Nicolò De Sabbata, Theodore R. Sumers, and Thomas L. Griffiths. 2024. Rational metareasoning for large language models. *CoRR*.

Rylan Schaeffer, Brando Miranda, and Sanmi Koyejo. 2023. Are emergent abilities of large language models a mirage? In *NeurIPS*.

Aman Sharma and Paras Chopra. 2025. Think just enough: Sequence-level entropy as a confidence signal for LLM reasoning. *CoRR*.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *ICLR*.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: language agents with verbal reinforcement learning. In *NeurIPS*.

Edward H Simpson. 1951. The interpretation of interaction in contingency tables. *Journal of the Royal Statistical Society*.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2024. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *CoRR*.

Linwei Tao, Yi-Fan Yeh, Minjing Dong, Tao Huang, Philip Torr, and Chang Xu. 2025. Revisiting uncertainty estimation and calibration of large language models. *CoRR*.

Surat Teerapittayanon, Bradley McDanel, and H. T. Kung. 2017. Branchynet: Fast inference via early exiting from deep neural networks. *CoRR*.

James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. 2018. Fever: a large-scale dataset for fact extraction and verification. In *NAACL*.

Jingwei Wang, Jianshan He, Weidi Xu, Ruopeng Li, and Wei Chu. 2023. Learning to discover various simpson's paradoxes. In *KDD*.

Zehong Wang, Fang Wu, Hongru Wang, Xiangru Tang, Bolian Li, Zhenfei Yin, Yijun Ma, Yiyang Li, Weixiang Sun, Xiusi Chen, and Yanfang Ye. 2026. Why reasoning fails to plan: A planning-centric analysis of long-horizon decision making in LLM agents. *CoRR*.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *EMNLP*.

Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2022. Webshop: Towards scalable real-world web interaction with grounded language agents. In *NeurIPS*.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023a. Tree of thoughts: Deliberate problem solving with large language models. In *NeurIPS*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023b. React: Synergizing reasoning and acting in language models. In *ICLR*.

Jiajie Zhang, Nianyi Lin, Lei Hou, Ling Feng, and Juanzi Li. 2025. Adaptthink: Reasoning models can learn when to think. In *EMNLP*.

Jiaxin Zhang, Prafulla Kumar Choubey, Kung-Hsiang Huang, Caiming Xiong, and Chien-Sheng Wu. 2026. Agentic uncertainty quantification. *CoRR*.

Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. 2024. Language agent tree search unifies reasoning, acting, and planning in language models. In *ICML*.

Yubo Zhu, Dongrui Liu, Zecheng Lin, Wei Tong, Sheng Zhong, and Jing Shao. 2025. The LLM already knows: Estimating llm-perceived question difficulty via hidden representations. In *EMNLP*.

A Extended Related Work

A.1 Adaptive Compute: Reasoning vs. Agent Settings

The adaptive test-time compute literature spans two distinct settings. In the *reasoning* setting, methods optimize single-turn chain-of-thought generation on homogeneous benchmarks (MATH, GSM8K, GPQA). Signal-based approaches use token-level confidence or entropy to decide when to extend thinking (Lee et al., 2025; Huang et al., 2026; Jin et al., 2026; Sharma and Chopra, 2025; Muennighoff et al., 2025; Han et al., 2025). RL-based methods learn think/no-think policies through reinforcement learning (Zhang et al., 2025; Fang et al., 2025; Aggarwal and Welleck, 2025). Hidden-state probing methods train lightweight classifiers on internal representations to estimate task difficulty (Liu et al., 2025; Zhu et al., 2025), sometimes assuming a universal U-shaped entropy pattern that our cross-environment evidence disproves. Because reasoning benchmarks share similar task semantics, signal direction happens to be consistent across tasks, and the fixed-direction assumption is never tested.

In the *agent* setting, methods operate in multi-step interactive environments with external optimizers. Vote-based methods (Lee et al., 2026; Jin et al., 2026) sample multiple candidate actions and trigger evaluation when disagreement is high. Verbalized-uncertainty methods (Zhang et al., 2026) use self-reported confidence for agent reflection but assume fixed direction. Episode-level approaches (Paglieri et al., 2025) learn when to invoke a planner, and agentic RL methods (Dong et al., 2025) use entropy-based adaptive rollout at tool-call steps. All agent-setting methods inherit the fixed-direction assumption from the reasoning literature without verifying it. Our work is the first to systematically study signal semantics across heterogeneous agent environments, where we find that the direction reverses.

A.2 Adaptive Computation in Neural Networks

The idea of adapting computation to input difficulty predates LLMs. Adaptive Computation Time (Graves, 2016) allows RNNs to learn how many computational steps to take per input. Early exit methods (Teerapittayanon et al., 2017; Elhoushi et al., 2024) attach classifiers at intermediate transformer layers and exit early for easy in-

puts. Mixture-of-Experts architectures (Shazeer et al., 2017; Raposo et al., 2024) activate only a subset of parameters per token, adapting capacity rather than depth. These methods adapt computation within a single forward pass. Our work operates at a different granularity: we adapt whether to invoke an external optimizer at each decision step, and the key challenge is not input difficulty but the *direction* of the signal–utility relationship.

A.3 LLM Confidence Estimation and Calibration

Kadavath et al. (2022) show that larger models are well-calibrated on multiple-choice tasks and can estimate P(True) for their own outputs. Tao et al. (2025) and Heo et al. (2025) find that calibration quality varies substantially across task types, providing converging evidence that uncertainty semantics are domain-dependent. Abbasi Yadkori et al. (2024) propose decoupling epistemic and aleatoric uncertainty in LLM outputs, a distinction conceptually related to our Type I/Type D decomposition. Our work extends these findings: not only does calibration *quality* vary across tasks, but the *direction* of the uncertainty–utility relationship reverses entirely.

A.4 Metareasoning and Value of Computation

Rational metareasoning (Russell and Wefald, 1991; Horvitz, 1987) formalizes the value of computation (VOC) as a guide for allocating cognitive resources. The classical framework assumes VOC ≥ 0 because an agent can always ignore computation results. Lieder and Griffiths (2020) generalize this to resource-rational analysis, modeling cognition as optimal use of limited computational resources. Sabbata et al. (2024) apply metareasoning to LLMs, analyzing when to extend chain-of-thought reasoning. Our work shows that the classical VOC ≥ 0 assumption is violated in the agent setting; we formalize this in Appendix B.3.

A.5 Simpson's Paradox in Machine Learning

Simpson's paradox (Simpson, 1951; Pearl, 2022) occurs when a trend present in subgroups reverses upon aggregation. In machine learning, this phenomenon has been documented in fairness (Wang et al., 2023), causal inference, and treatment effect estimation. Our Two-Source Model identifies a new instance: within Type I states, entropy negatively predicts utility; within Type D states, entropy positively predicts utility; but the marginal correlation depends on the mixture proportion $p_I(\mathcal{E})$, which varies across environments. This makes direction reversal a structural consequence of aggregating heterogeneous uncertainty sources.

B Theoretical Material

This appendix collects the theoretical material referenced in Section 3: the proof of Proposition 1, the wrong-direction damage quantification, and the VOC negativity analysis.

B.1 Proof of Proposition 1 (Necessity of Direction Discovery)

Proposition (Restated)**.** *Suppose there exist two environments $\mathcal{E}_1, \mathcal{E}_2$ with opposite true directions: $d^*(\mathcal{E}_1) = +1$ and $d^*(\mathcal{E}_2) = -1$. Then no fixed-direction gate g_d can simultaneously satisfy $\mathrm{SR}(g_d, \mathcal{E}_1) \geq \mathrm{SR}(\mathrm{base}, \mathcal{E}_1)$ and $\mathrm{SR}(g_d, \mathcal{E}_2) \geq \mathrm{SR}(\mathrm{base}, \mathcal{E}_2)$.*

Proof. Define a fixed-direction gate as $g_d(s) = \mathbf{1}[d \cdot \sigma(s) > \theta]$ for $d \in \{+1, -1\}$ and threshold θ.

Define $d^*(\mathcal{E}) = +1$ if $\mathrm{Corr}(\sigma(s), U(T, s)) > 0$ in $\mathcal{E}$ (Type D), and $d^*(\mathcal{E}) = -1$ if $\mathrm{Corr}(\sigma(s), U(T, s)) < 0$ (Type I).

Lemma (Wrong-direction harm). If $d \neq d^*(\mathcal{E})$, then $\mathrm{SR}(g_d, \mathcal{E}) \leq \mathrm{SR}(\mathrm{base}, \mathcal{E}) - \delta(\mathcal{E})$ where $\delta(\mathcal{E}) > 0$ depends on signal strength $|\rho(\mathcal{E})|$.

Proof of Lemma: By definition of d^*, states with high $d \cdot \sigma(s)$ are states where $U < 0$. The gate triggers T at these states, causing the agent to adopt worse actions. The base policy, which never triggers T, avoids this systematic harm.

Main argument:

Case 1: $d = +1$. In $\mathcal{E}_2$ where $d^* = -1$, $d \neq d^*(\mathcal{E}_2)$. By Lemma: $\mathrm{SR}(g_{+1}, \mathcal{E}_2) \leq \mathrm{SR}(\mathrm{base}, \mathcal{E}_2) - \delta(\mathcal{E}_2) < \mathrm{SR}(\mathrm{base}, \mathcal{E}_2)$. Constraint violated.

Case 2: $d = -1$. In $\mathcal{E}_1$ where $d^* = +1$, by symmetric argument, constraint violated.

Since $d \in \{+1, -1\}$ and both cases lead to violation, no fixed d satisfies both constraints simultaneously. □

B.2 Wrong-Direction Damage Quantification

The proof above bounds wrong-direction harm by $\delta(\mathcal{E})$ but does not specify its magnitude. Table 3 reports the empirically observed damage from the

BSW ablation, showing that $\delta(\mathcal{E})$ scales with signal strength $|\rho(\mathcal{E})|$.

Table 3: Wrong-direction damage scales with signal strength.

Environment	Correct SR	Wrong SR	Δ SR	$\lvert\rho\rvert$ (strongest)
FEVER	49.8%	13.0%	−36.8	0.619
HotpotQA	95.2%	58.2%	−37.0	0.494
WebShop	43.8%	20.6%	−23.2	0.444
TWExpress	99.0%	99.0%	0.0	0.290
Plancraft	23.3%	25.5%	+2.2	0.016
APPS	73.0%	79.5%	+6.5	0.339

Among environments with strong signal–utility coupling ($|\rho| > 0.15$), wrong direction causes substantial SR degradation (FEVER −36.8 pp, HotpotQA −37.0 pp, WebShop −23.2 pp). In weak-signal environments (Plancraft $|\rho|$=0.016), direction has negligible effect. Two exceptions—TWExpress (rollout-safe, Δ=0) and APPS (+6.5 pp)—reflect ceiling effects and near-oracle baselines respectively; their gate decisions differ minimally regardless of direction.

B.3 VOC Non-Negativity Scope

In the classical VOC framework (Russell and Wefald, 1991), $\mathrm{VOC}(T, s) = \mathbb{E}[\max(V(a_T), V(a_{\text{base}}))] - V(a_{\text{base}}) \geq 0$ because the agent can *disregard* the computation result if unfavorable. Our setting violates this: when the agent uses an evaluator/optimizer T, the evaluator's assessment *is* the agent's decision. The agent cannot separately evaluate the evaluator's output because the evaluator *is* the evaluation mechanism.

Under evaluator-executor identity, VOC can be negative: $\mathrm{VOC}(T, s) = \mathbb{E}[V(a_T(s))] - V(a_{\text{base}}(s))$. If T systematically selects worse actions in Type I states, $\mathrm{VOC} < 0$ for those states. This explains why wrong-direction gating is not merely wasteful ($\mathrm{VOC} = 0$) but actively harmful ($\mathrm{VOC} < 0$).

C Experimental Setup

C.1 Environment Specifications

Table 4 summarizes the 6 environments with their base performance, optimizer headroom, and optimizer type. Per-action evaluation generates K=5 candidate actions and selects the best; K-variant sampling generates K=3 code solutions and selects a passing one; LLM-Propose-K has the LLM propose K=3 action sequences, simulates each, and selects the highest-reward trajectory.

Table 4: Environment setup. Base/Always: SR without/with optimizer. Δ: rollout headroom.

Environment	Base SR	Always SR	Δ	Optimizer T
HotpotQA	49.0%	97.0%	+48.0	Per-action eval
WebShop	7.2%	43.0%	+35.8	LLM-Propose-K
FEVER	37.0%	99.8%	+62.8	Per-action eval
TWExpress	67.5%	99.3%	+31.8	Per-action eval
Plancraft	29.8%	22.8%	−7.0	Per-action eval
APPS	60.5%	79.5%	+19.0	K-variant sampling

C.2 Detailed Baseline Comparison

Table 5 provides a detailed comparison of all baselines evaluated in this paper. All six methods share a common assumption: a fixed mapping from signal value to compute need.

C.3 EAAG Implementation Details

This subsection expands on the implementation details deferred from Section 4.3. Algorithm 1 gives the full pipeline; the paragraphs that follow describe each component.

Algorithm 1 EAAG: Environment-Aware Adaptive Gating

Require: Environment $\mathcal{E}$, agent policy π, optimizer T, universal feature set ϕ_{univ}, LLM $\mathcal{M}$, exploration budget N_{explore}, exploration rate $\varepsilon_{\text{explore}}$, gate threshold τ
Ensure: Deployed gate $g : \mathcal{S} \to \{0, 1\}$
// Phase 1: Explore
$\mathcal{D} \leftarrow \emptyset$
for episode $= 1, \ldots, N_{\text{explore}}$ **do**
  Run π in $\mathcal{E}$; at each step t, invoke T with probability $\varepsilon_{\text{explore}}$ independent of state
  Record (s_t, U_t) pairs into $\mathcal{D}$
end for
// Phase 2: Reason
$(p_{\mathcal{E}}, \mathcal{H}) \leftarrow \mathcal{M}(\mathrm{summary}(\mathcal{D}))$ ▷ env profile + feature hypotheses
$\phi_{\text{cand}} \leftarrow \phi_{\text{univ}} \cup \mathrm{extract}(\mathcal{H})$
Recompute $\phi_{\text{cand}}(s)$ for all $s \in \mathcal{D}$
// Phase 3: Learn
Standardize ϕ_{cand} on $\mathcal{D}$
Select C via 5-fold CV on held-out log-loss
Fit ℓ_1-regularized logistic regression: $\mathbf{w}, b \leftarrow \arg\min \mathcal{L}_{\text{logistic}}(\mathbf{w}, b; \mathcal{D}) + \frac{1}{C}\|\mathbf{w}\|_1$
$\phi \leftarrow$ features with non-zero w_i
$g(s) \leftarrow \mathbf{1}[\sigma(\mathbf{w}^\top \phi(s) + b) > \tau]$
return g

Exploration budget. Phase 1 runs for $N_{\text{explore}} = 50$ episodes with $\varepsilon_{\text{explore}} = 0.5$. Sensitivity to N_{explore} is reported in §F.8; SR is stable for $N_{\text{explore}} \geq 30$.

Feature standardization. Before fitting in Phase 3, all features in ϕ_{cand} are standardized to

Table 5: Detailed comparison of baselines. All methods assume a fixed signal–utility direction. †Requires a separate labeled dataset for calibration.

Method	Signal	Direction	Granularity	Mechanism
CATTS (Lee et al., 2026)	Vote entropy + margin	High disagreement → trigger	Per-step	K=5 forward passes, arbiter on disagreement
SEAG† (Lee et al., 2025)	Mean token confidence	Low conf. → search	Per-problem	Platt scaling, search depth allocation
CaTS† (Huang et al., 2026)	Calibrated confidence	Low conf. → generate	Per-attempt	Self-calibration, Best-of-N early stopping
CoRefine† (Jin et al., 2026)	Conv1D confidence	Low conf. → refine	Per-problem	Halt / re-examine / redirect controller
s1_budget (Muennighoff et al., 2025)	Token budget	Fixed budget → stop	Per-problem	Budget-constrained generation
AUQ (Zhang et al., 2026)	Verbalized uncertainty	High uncert. → reflect	Per-step	Uncertainty-aware memory + reflection
EAAG	**Multi (auto)**	**Learned**	**Per-step**	Explore + reason + LASSO direction learning

zero mean and unit variance on $\mathcal{D}$. This ensures that the interpretable weight signs discussed in Section 4.3 are not confounded by scale differences across heterogeneous features (e.g., `step_count` and `token_entropy` live on very different scales).

Sparse solver. We induce sparsity using ℓ_1-regularized logistic regression (`scikit-learn`'s `LogisticRegression` with `liblinear` solver). The regularization strength C is selected by 5-fold cross-validation over the grid $C \in \{0.01, 0.03, 0.1, 0.3, 1.0, 3.0, 10.0\}$, optimizing held-out log-loss on $\mathcal{D}$. Features with zero coefficient after fitting are dropped from the deployed gate.

Gate threshold. The decision threshold τ in Eq. (7) is fixed to 0.5 for all environments. Sensitivity across $\tau \in [0.3, 0.7]$ is reported in §F.8.

Online adaptation. During deployment, $\varepsilon_{\text{deploy}}$ decays from 0.1 to 0 over the first 100 episodes, and the gate is refit every $k = 30$ episodes using the accumulated ε-greedy samples. Refitting reuses the same solver configuration as the initial training, so no additional hyperparameter tuning is required at deployment time.

C.4 LLM Reasoning Prompts

The full LLM prompt template is provided below. We use WebShop as the example, where LLM features contribute most meaningfully.

```
You are an AI agent that
has been exploring an
interactive environment.
During exploration, you
sometimes performed ``rollouts''
(additional computation)
to improve your decisions.
Sometimes the rollout was useful
(positive utility), sometimes it
was not.

Your task: Analyze the patterns
in your experience and write a
Python function that extracts
features from the state text
that could predict whether a
rollout would be useful.

[Exploration statistics
and representative examples
provided]

Requirements: Return a dict
with string keys and float
values. Use only Python
standard library. Extract
exactly 5 features based on
the patterns you observe. Focus
on features that DIFFER between
useful and not-useful rollout
cases.
```

C.5 Computational Cost Breakdown

Table 6 and Figure 3 break down the computational cost per method. Cost is reported as ×base: total deployment tokens (base proposer + gate overhead + rollout calls) normalized by base_only tokens per episode. This metric captures hidden per-step overhead that rollouts-per-episode misses: CATTS incurs K=5 forward passes *every step* regardless of the gating decision, and AUQ issues a confidence query every step.

Table 7 reports the per-environment token cost constants used to compute ×base. All constants are empirically measured except FEVER, which is estimated from HotpotQA (same optimizer type).

Table 6: Computational cost breakdown per method. Phase 1 = pre-deployment calibration or exploration. Gate OH = additional LLM calls per step beyond the base proposer. Cost (×base) = total deployment tokens normalized by base_only tokens per episode, averaged across 6 environments. †Requires calibration data.

Method	Phase 1	Type	Gate OH/step	OH source	Avg ×base
base_only	0	—	0	—	1.00
always_trigger	0	—	0	—	5.97
CATTS	0	optional	$+K$=5 calls	Voting (forward pass)	6.32
SEAG†	200	calibration	0	Reads logprob	5.21
CoRefine†	200	calibration	0	Reads entropy	5.16
CaTS†	200	calibration	0	Reads confidence	7.75
AUQ	0	optional	+1 query	Confidence question	5.90
s1_budget	0	—	0	—	2.38
EAAG	**50**	exploration	**0**	**Learned gate**	**5.58**

Table 7: Token cost constants per environment. C_{base}: tokens per base proposer step. C_{rollout}: tokens per rollout trigger. C_{vote}: CATTS K=5 voting tokens per step. ‡Estimated from HotpotQA (same optimizer type).

Environment	C_{base}	C_{rollout}	C_{vote}	Base tokens/ep	
HotpotQA	216	7,743	1,063	1,348	
WebShop	705	9,089	3,385	9,870	
FEVER‡	250	7,500	1,250	1,438	estimated
TWExpress	524	8,002	2,620	13,660	
Plancraft	1,120	10,651	5,598	13,418	
APPS	840	3,306	4,198	2,486	

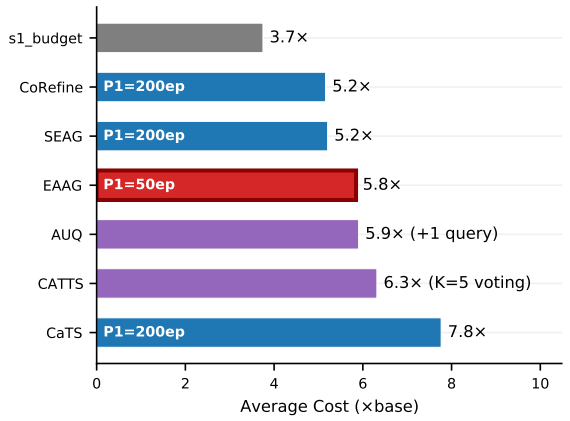

Figure 3: Computational cost breakdown (×base). EAAG (red border) requires only 50 exploration episodes with zero per-step gate overhead. CATTS's K=5 voting overhead dominates its cost even when rollouts are rarely triggered (e.g., APPS: C_{vote}=4,198 > C_{rollout}=3,306).

D Full Results

Table 8 reports SR and Cost (×base) for all methods across 6 environments and 3 backbones. Cost includes all deployment LLM calls (base proposer, gate overhead, rollouts) normalized by base_only cost. Several patterns are consistent across backbones.

Qwen3-4B. EAAG achieves the highest SR in 4 of 6 environments (HotpotQA, WebShop, TWExpress, APPS). On WebShop, it matches always_trigger SR (43.8% vs. 43.0%) at less than half the cost (2.50× vs. 5.56×). The ×base metric reveals hidden costs: CATTS on APPS costs 6.00× base (due to K=5 voting) despite barely triggering rollouts (due to K=5 per-step voting), while EAAG costs only 2.61× at 12.2 pp higher SR. FEVER is the main exception, where EAAG (49.8%) underperforms CaTS (50.2%); this failure mode is analyzed in §6.

Phi-3.5-mini. Fixed-direction baselines degrade sharply on this backbone. On FEVER, all six baselines score below 24% (base: 7.2%), while EAAG reaches 16.5%. On WebShop, EAAG (57.3%) exceeds the best baseline (CaTS, 41.7%) by 15.6 pp.

Llama-3.1-8B. Direction reversal interacts with model capacity. On TWExpress, CaTS collapses to near-base performance (35.8% vs. base 36.5%) because it never triggers, while EAAG achieves 94.8%. On HotpotQA and APPS, most baselines perform comparably to EAAG because the high base SR leaves little room for gating to help.

E Signal and Feature Analyses

This appendix collects empirical analyses of signal identity and signal sufficiency referenced in Section 3.1.

E.1 Per-Environment Feature Selection

The LASSO coefficient matrix shows that `step_count` is informative in 4 of 5 environments with significant signal. WebShop is the exception, relying on `num_available_actions` and task-specific features (`price_mentioned`, `action_is_click`) reflecting its choice-dominated structure.

E.2 LLM-Generated Features

Table 9 reports the features generated by the LLM reasoning step (Phase 2) for each environment.

Table 8: Full results across 6 environments and 3 backbones. SR (%) and Cost (×base, total deployment tokens normalized by base_only). **Bold**: best SR per environment (excluding oracle). †: methods requiring calibration data. Cost includes all LLM calls: base proposer, gate overhead (CATTS K=5 voting, AUQ confidence query), and rollout calls.

	HotpotQA		**WebShop**		**FEVER**		**TWExpress**		**Plancraft**		**APPS**	
Method	SR	×b	SR	×b	SR	×b	SR	×b	SR	×b	SR	×b
Qwen3-4B-Instruct												
base_only	49.0	1.00	7.2	1.00	37.0	1.00	67.5	1.00	29.8	1.00	60.5	1.00
always_trigger	97.0	10.63	43.0	5.56	99.8	7.87	99.3	2.15	22.8	6.13	79.5	3.65
CaTS†	93.2	10.54	30.5	3.45	50.2	25.48	96.7	1.37	22.3	4.14	66.2	1.61
SEAG†	67.5	5.34	28.0	2.84	49.3	17.19	97.3	1.60	24.8	2.52	66.0	1.75
CoRefine†	68.2	5.28	27.5	2.78	49.8	17.19	97.5	1.57	22.8	2.44	67.5	1.70
CATTS	68.3	10.53	16.0	5.55	34.2	6.46	97.5	2.83	25.0	6.53	60.8	6.00
AUQ	97.0	10.29	35.7	5.83	40.7	8.04	95.5	1.35	24.2	6.64	64.7	3.28
s1_budget	97.0	6.27	7.8	1.91	46.2	9.21	95.0	1.02	18.3	2.02	69.0	2.15
BSW	58.2	17.35	20.6	5.77	63.0	23.37	99.0	2.04	25.5	5.33	79.5	2.66
EAAG	**95.2**	8.02	**43.8**	2.50	49.8	16.51	**99.0**	1.81	23.3	3.63	**73.0**	2.61
Phi-3.5-mini-instruct												
base_only	28.3	1.00	3.5	1.00	7.2	1.00	66.7	1.00	13.7	1.00	27.0	1.00
always_trigger	98.8	5.77	53.5	3.71	61.0	17.86	97.8	3.07	14.5	6.44	79.5	2.28
CaTS†	68.3	5.26	41.7	1.52	19.8	14.51	68.2	1.00	13.5	1.00	30.5	3.55
SEAG†	88.3	7.82	36.5	1.43	13.5	5.61	94.5	1.58	14.7	2.29	27.8	1.59
CoRefine†	87.7	7.83	35.8	1.44	13.7	5.73	94.7	1.56	15.2	2.24	28.5	1.60
CATTS	42.0	7.98	28.7	4.86	12.7	7.65	86.7	4.00	13.5	5.96	27.7	6.10
AUQ	31.2	2.37	7.0	2.11	8.5	3.18	94.5	1.82	12.7	2.24	27.2	2.03
s1_budget	94.2	5.26	3.0	1.89	23.0	8.45	95.7	1.05	13.7	1.73	34.5	1.93
EAAG	**92.3**	5.38	**57.3**	2.56	16.5	10.44	**96.7**	1.83	13.8	1.33	**36.8**	2.87
Llama-3.1-8B-Instruct												
base_only	46.3	1.00	1.2	1.00	13.2	1.00	36.5	1.00	31.7	1.00	60.2	1.00
always_trigger	99.5	7.09	42.8	4.33	62.8	17.97	99.5	3.48	18.7	7.60	79.5	3.66
CaTS†	96.2	5.21	36.0	2.51	56.3	14.82	35.8	1.00	22.8	3.42	60.2	2.21
SEAG†	96.0	9.82	27.5	1.86	53.3	12.57	76.5	2.85	21.8	2.70	60.2	2.41
CoRefine†	96.2	9.42	27.5	1.82	53.5	12.64	76.0	1.56	21.5	2.69	60.2	2.37
CATTS	96.0	11.11	1.3	4.74	14.2	17.00	49.3	4.11	27.0	7.31	52.3	6.70
AUQ	97.0	8.54	37.8	4.54	52.8	25.08	63.2	1.58	27.0	8.21	60.0	6.11
s1_budget	96.5	6.08	1.2	1.87	13.8	8.56	76.1	1.05	30.3	2.53	72.5	2.33
EAAG	**95.5**	7.07	**41.7**	1.68	34.7	18.69	**94.8**	2.55	27.0	2.49	59.7	4.29

The LLM receives exploration statistics and representative positive/negative examples, then produces a Python function extracting 5 task-specific features (the full prompt template is in §C.4). LASSO subsequently selects or discards each feature based on its predictive value for rollout utility.

Table 9: LLM-generated features per environment. Each environment receives 5 LLM-proposed features; LASSO selects those with non-zero coefficients. Environments with richer task-specific structure (WebShop, TWExpress) retain more LLM features.

Environment	Sel./Gen.	LASSO-Selected LLM Features
WebShop	4/5	`price_mentioned`, `action_is_click`, `step_early`, `instruction_keyword_count`
TWExpress	4/5	`text_length`, `closed_ratio`, `action_look_around`, `already_open`
HotpotQA	3/5	`has_question`, `entity_count`, `fact_keyword_count`
FEVER	2/5	`text_length_normalized`, `has_claim`
APPS	1/5	`action_type`
Plancraft	0/5	(none selected)

Environments with rich task-specific structure

benefit most from LLM features: WebShop's choice-dominated interface is well captured by `price_mentioned` and `action_is_click`, while TWExpress's exploration mechanics are reflected in `closed_ratio` and `action_look_around`. In contrast, Plancraft's rollout-harmful nature means no LLM feature improves prediction—LASSO correctly filters all five proposals. This pattern aligns with the LLM ablation results (Table 2): the environment where LLM features contribute most to SR (FEVER, +9.1 pp) is precisely where the LLM discovers unique signals (`has_claim`) that the universal feature pool lacks.

The following listing shows the complete LLM output for WebShop (seed 42), the environment with the most LLM features retained.

```
def extract_features(state_text,
step_count, action_text):
  import re
  features = {}
  # Instruction keyword density
  keywords = ["gluten free",
"protein",
    "price lower than", "size",
"color"]
  matches = sum(1 for kw in
keywords
    if kw in state_text.lower())
  features['instruction_keyword_count']
= (
    float(matches) /
len(keywords))
  # Early-step indicator
  features['step_early'] = (
    1.0 if step_count <= 2 else
0.0)
  # Click vs. search action
  features['action_is_click'] = (
    1.0 if
action_text.startswith("click[")
    else 0.0)
  # Price constraint presence
  price_matches = len(re.findall(
    r"price lower
than|price:|\$",
    state_text, re.IGNORECASE))
  features['price_mentioned'] = (
    float(price_matches) / 5.0)
  # Filter complexity
  filter_kws =
["size","color","flavor",
    "price","rating"]
  features['filter_count'] = sum(
    1 for kw in filter_kws
    if kw in state_text.lower())
/ 5.0
  return features
```

E.3 AUC Hierarchy: Single vs. Multi-Signal Gates

Table 10 compares gate families on held-out data. Single entropy is near chance (AUC≈0.50) in every environment; multi-signal logistic gates reach 0.74–0.92; and hidden-state probes reach 0.79–0.99. The bottleneck is signal *combination*, not gate *capacity*.

Table 10: AUC by gate family. Single entropy is near chance; multi-signal gates close most of the gap to hidden-state probes. FEVER and TWExpress are omitted because their short episodes yield insufficient held-out data for reliable AUC estimation.

Gate family	HotpotQA	APPS	WebShop	Plancraft
Single entropy	0.50	0.56	0.50	0.50
Best single signal	0.78	0.78	0.90	0.74
Multi-signal LR	0.85	0.76	0.92	0.74
Hidden-state probe	0.87	0.79	0.99	0.95

F Robustness and Additional Experiments

This appendix collects Two-Source Model verification data, robustness checks for direction reversal, and additional experimental analyses.

F.1 Temporal Dynamics (P1)

Table 11 reports the full early/late ρ data referenced in Figure 2. The Two-Source Model predicts that $\rho(\text{entropy}, U)$ should decrease from early to late steps within each episode, as late-step uncertainty increasingly reflects the residual Type I component after information-gathering attempts.

Table 11: Temporal shift in ρ(entropy, U). All environments show ρ shifting more negative from early to late steps, consistent with P1. Plancraft omitted ($|\rho|$=0.016, too weak for meaningful comparison).

Env	Phase	ρ	95% CI	n	p	Shift
HotpotQA	early	−0.176	[−0.260, −0.091]	509	<0.001	Δ=−0.261
	late	−0.437	[−0.500, −0.370]	699	<0.001	
APPS	early	+0.102	[−0.013, +0.209]	290	0.084	Δ=−0.246
	late	−0.144	[−0.281, +0.001]	199	0.043	
WebShop	early	+0.285	[+0.222, +0.346]	600	<0.001	Δ=−0.291
	late	−0.006	n.s.	473	0.895	
FEVER	early	+0.054	n.s.	200	0.446	Δ=+0.024
	late	+0.078	n.s.	82	0.486	
TWExpress	early	+0.161	—	422	<0.001	Δ=−0.153
	late	+0.008	n.s.	376	0.877	

The prediction holds across 4 of 5 testable environments: HotpotQA shows the strongest shift (Δ=−0.261), with late-step entropy reflecting deeper information poverty after failed

evidence retrieval. WebShop's strong positive early-phase signal (ρ=+0.285) vanishes in the late phase (ρ=−0.006, n.s.), consistent with the choice-dominated structure being resolved early. In APPS, ρ reverses from marginally positive (+0.102) to negative (−0.144), as late-step decision complexity gives way to residual information poverty.

FEVER is the exception: both phases show weak, non-significant positive ρ, consistent with FEVER's short episodes (median 8 steps) concentrating most rollout value at step 0–1, leaving insufficient late-step variation for a meaningful shift.

F.2 Signal Identity Alignment (P3)

Table 12: P3 Signal identity alignment. The strongest signal in each environment matches the Two-Source Model prediction: information-sufficiency proxies dominate in Type I environments; decision-complexity proxies dominate in mixed/Type D.

Environment	Type	Strongest Signal	$\lvert\rho\rvert$	Interpretation
HotpotQA	Info-Poverty	step_count	0.494	Info accumulation
FEVER	Info-Poverty	step_count	0.619	Info accumulation
TWExpress	Info-Poverty	step_count	0.477	Exploration depth
WebShop	Mixed	num_available_actions	0.444	Decision-space size
APPS	Decision-Diff	step_count	0.339	Debugging depth
Plancraft	Weak (harmful)	has_output	0.162	Rollout harmful

F.3 Controlled Information Manipulation

To provide stronger evidence that information structure drives signal semantics, we manipulate information availability within HotpotQA itself. *InfoPoor* restricts search to the first sentence; *InfoRich* injects gold evidence at the start. Table 13 shows the results. In InfoPoor, `step_count` dominates (ρ=−0.608) while entropy is weak (ρ=+0.119); in InfoRich, entropy becomes the dominant signal (ρ=+0.311) while `step_count` weakens (ρ=−0.147). This demonstrates that the *identity* of the most informative signal, not just its direction, is a function of the environment's information structure. EAAG adapts to both conditions without manual reconfiguration.

Table 13: Controlled information manipulation on HotpotQA. Manipulating information availability shifts which signal dominates, confirming the Two-Source Model's prediction.

Condition	Base SR	ρ(entropy)	ρ(step_count)	EAAG SR	n
Original	49.0%	−0.041	−0.494	95.2%	1117
InfoPoor	44.0%	+0.119	**−0.608**	94.0%	317
InfoRich	82.0%	**+0.311**	−0.147	88.3%	221

F.4 Multi-Backbone Verification

We validate our core findings on three backbones from three vendors: Qwen3-4B-Instruct (Alibaba, 4B), Phi-3.5-mini-instruct (Microsoft, 3.8B), and Llama-3.1-8B-Instruct (Meta, 8B) across all 6 environments. Table 8 reports full results.

Figure 4 visualizes the direction variation, with gold borders marking sign flips. On Phi-3.5 FEVER: most fixed-direction baselines score near or below base_only (7.2% base → 8.5–23%), demonstrating that the fixed-direction assumption is brittle when the model changes (Figure 5).

F.5 Stratified Analysis: Controlling for Trajectory Length

Trajectory length varies across environments and correlates with both entropy and utility. To rule out length as a confounder, we stratify by step count and recompute $\rho(\text{entropy}, U)$ within each stratum. Direction reversal persists within trajectory-length strata where utility variance is non-zero. In HotpotQA, ρ remains negative across all three strata (−0.18/−0.46/−0.42). The reversal is not an artifact of trajectory length.

F.6 Gate Complexity Ablation

To verify that the bottleneck is direction rather than gate complexity, we compare gate architectures with correct vs. wrong direction (Table 14).

Table 14: Gate complexity ablation on HotpotQA. Direction matters more than model capacity: a logistic gate with correct direction outperforms an MLP with wrong direction by 49.9 pp.

Gate	Direction	SR (%)	Δ
Logistic (5 feat)	Correct	95.2	+46.2
MLP (5 feat)	Correct	∼95	∼+46
Hidden state (2560-d)	Correct	∼95	∼+46
Logistic (5 feat)	Wrong	62.0	+13.0
MLP (5 feat)	Wrong	45.3	−3.7
No gate (base)	n/a	49.0	0

With correct direction, all architectures achieve ∼95% SR; the jump from logistic to MLP to hidden-state probe adds <1 pp. With wrong direction, higher capacity makes performance *worse* (MLP 45.3% < logistic 62.0%), because the more powerful gate more precisely targets harmful states. This confirms that signal direction consistently outweighs gate complexity, and explains why EAAG's minimal gate design suffices.

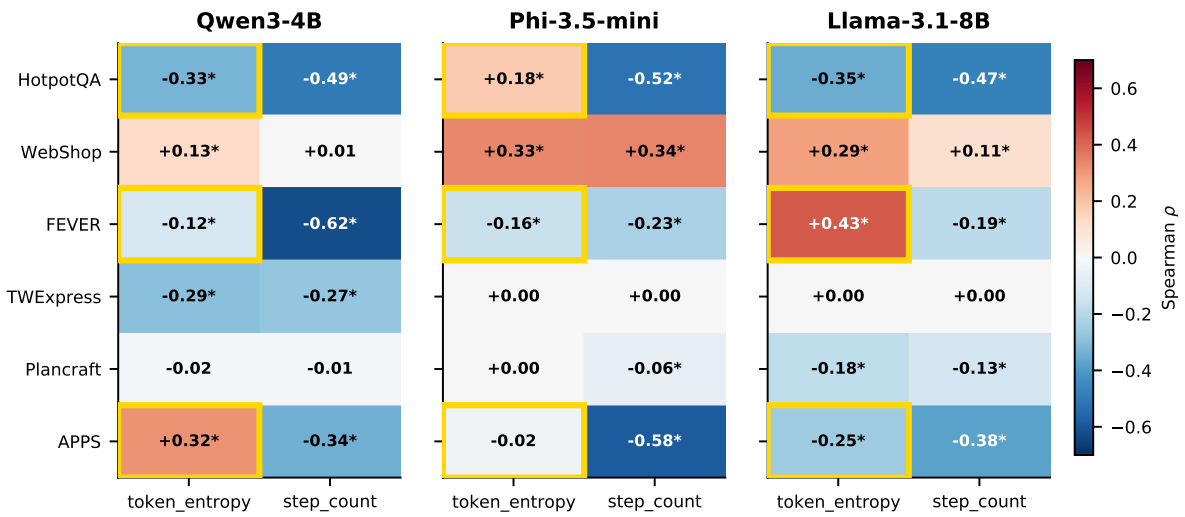

Figure 4: Multi-backbone signal direction heatmap. Gold borders mark cells where the sign of ρ flips across backbones with significant correlations. Entropy direction is model-dependent; structural signals (step_count) are more stable.

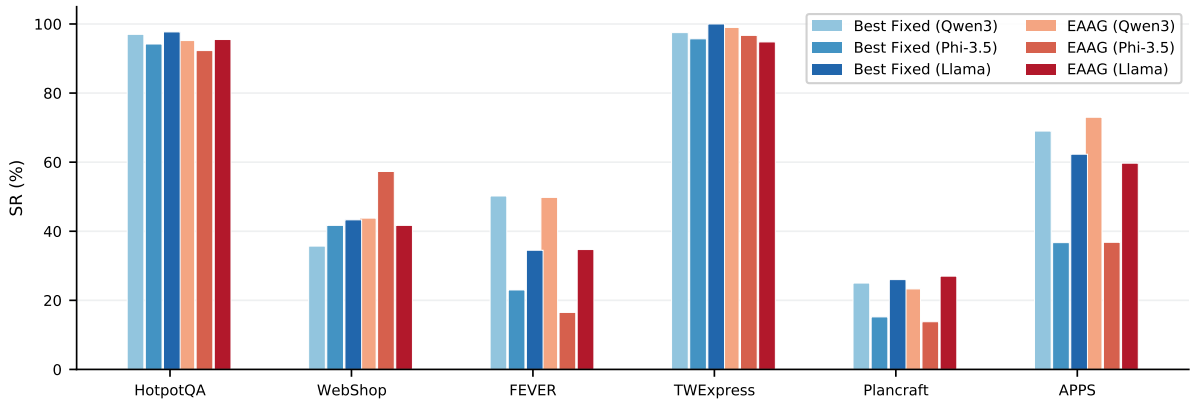

Figure 5: Cross-backbone comparison: EAAG (red) vs. best fixed-direction baseline (blue) across 3 backbones. Fixed baselines show large SR variance across backbones; EAAG adapts robustly.

F.7 Environment-Adaptive Trigger Behavior

Beyond direction and feature selection, EAAG's gating magnitude also adapts to each environment without explicit programming. Figure 6 shows the per-step trigger rate across all 6 environments, ordered by rollout headroom Δ. In rollout-safe TWExpress (Δ=+31.8), EAAG triggers aggressively (RR = 73%), consistent with the high headroom. In rollout-harmful Plancraft (Δ=−7.0), the trigger rate decays from 49% at step 0 to <20% in later steps, as the gate learns that late-step rollouts are increasingly harmful. WebShop and APPS show moderate, stable trigger rates. This full behavioral range emerges from the per-environment signed weights fitted in §4.3: the gate does not need an explicit headroom estimate, because signal direction already encodes whether the environment rewards or penalizes additional computation.

F.8 Hyperparameter Sensitivity

EAAG is robust to $N_{\text{explore}} \in [30, 100]$ (stable SR for $N_{\text{explore}} \geq 30$; marginal gains above 50) and gate threshold $\tau \in [0.3, 0.7]$.

G Broader Impact

This work addresses computational efficiency in LLM agent deployment. By reducing unnecessary optimizer invocations, EAAG lowers the energy and cost footprint of test-time compute without sacrificing task performance. The prescriptive framework (characterize information structure before designing gates) may help practitioners avoid deploying fixed-direction methods that ac-

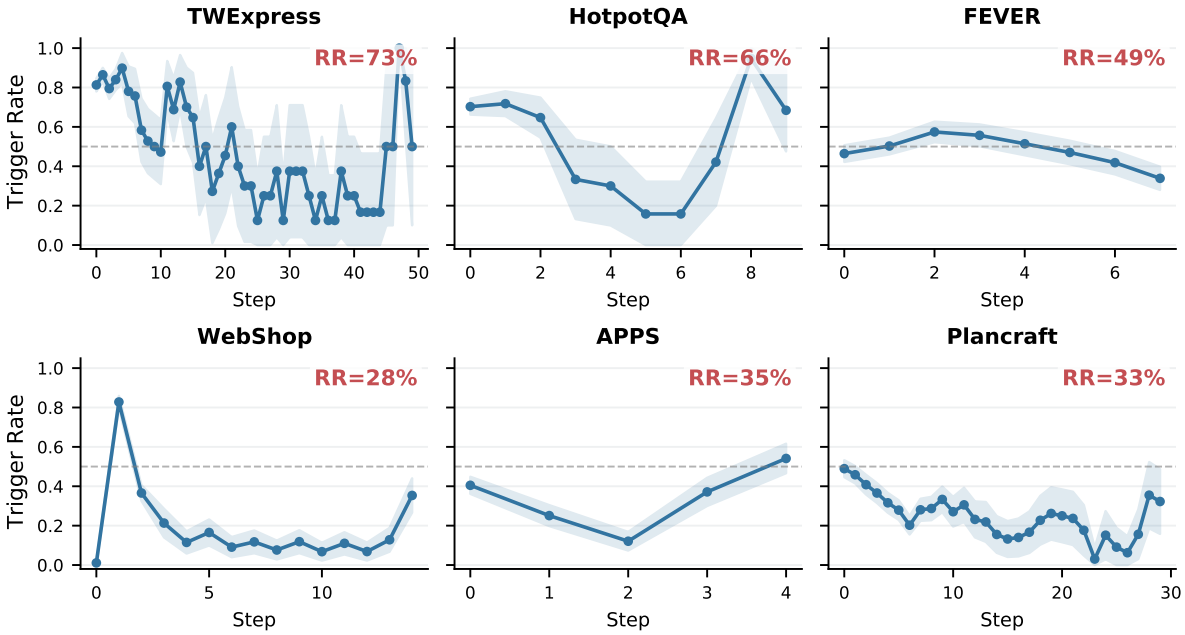

Figure 6: Environment-adaptive trigger behavior. Panels ordered by rollout headroom Δ. Solid line: per-step trigger rate with 95% CI band. Dashed gray: 50% reference. RR: overall trigger rate. EAAG's trigger rate ranges from 73% (TWExpress) to <20% (late-step Plancraft), emerging from direction learning alone.

tively harm performance in mismatched environments. We do not foresee direct negative societal impacts beyond those inherent to the LLM agents themselves.